

ROBOCUPJUNIOR RESCUE (LINE/MAZE) 2026

TEAM DESCRIPTION PAPER

B.Robots Seniors

Abstract

This abstract describes the autonomous rescue robot developed by Team B.Robots Seniors for the 2026 RoboCupJunior Rescue Maze Competition. The design prioritizes high reliability, modular hardware, and a deterministic software architecture. The main navigation controller, an Arduino Giga R1, executes an A* pathfinding algorithm on a weighted 3D tile graph to navigate obstacles and multi-floor ramps dynamically. A Raspberry Pi Compute Module 5 processes dual wide-angle camera streams using custom TensorFlow Lite models and HSV color scans to accurately identify victims.

A key innovation that sets our robot apart is the custom "Inter-Layer Connector." This system uses permanently mounted pin headers to replace messy wiring, enabling the rapid detachment and swap of entire hardware layers without unplugging individual wires. Furthermore, a custom-machined swivel axis ensures that all four wheels maintain constant ground contact over uneven terrain, which significantly improves sensor reliability.

On the software side, a strictly non-blocking C++ architecture maintains a rapid 23 ms control loop. During a run, an interactive touch display renders a live, rotating map for real-time diagnostics. A checkpoint snapshot system allows for seamless map and position restoration following manual interventions.

1) Introduction

a) Team

- *Paul Charusa (Team Captain)*

Paul serves as the captain for our team. His responsibilities include working on the design and the hardware of the robot.

- *Thomas Rauch*

Thomas is responsible for developing and implementing the AI-driven image processing systems used for victim detection on the Raspberry Pi CM5.

- *Vincent Rohkamm*

Vincent is responsible for the Mapping system and for the most parts the main program as well as the UI design for the Display.

- *Florian Wiesner*

Florian focuses on the programming of all the sensors and actors. He was also responsible for creating the driving functions and heavily involved in the main program.

2) Project Planning

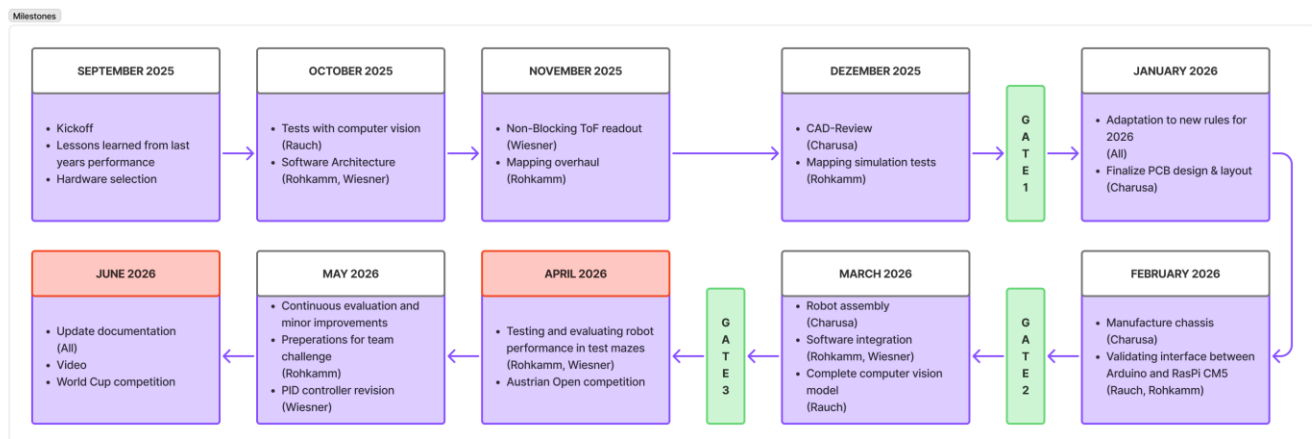
a) Overall Project Plan

To ensure success under competition constraints, we defined requirements based on the official RoboCup Junior Rescue Maze rules and personal findings over the last years.

ID	Technical Requirement / Constraint	Rule Reference	Justification & Strategy
REQ-01 (Size)	Maximum dimensions must be $\leq 20\text{cm} \times 20\text{cm} \times 20\text{cm}$	Rule 3.3.3	Maze tiles are $28\text{cm} \times 28\text{cm}$ with possible wall misalignments. A sub- 20cm footprint ensures critical safety clearance to prevent physical jamming during turns.
REQ-02 Vision	Victim detection and identification with an accuracy of $\geq 90\%$	Rule 3.6	Victims must be correctly classified to score maximum points. Requires two cameras linked to a RasPi CM5 with an TPU. Custom trained TensorFlow Lite models for rapid inference.
REQ-03 (Driving)	Smooth, deterministic multi-wall alignment and tactile obstacle handling. Loop time $\leq 25\text{ms}$	Rule 3.4 / Rule 3.5	The maze features obstacles and physical debris. The robot requires a combination of ToF sensors, gyro-sensors and physical micro-switch bumpers to handle navigation without blocking the control loop.
REQ-04 (Color)	Active floor color detection (Spectrometer) sampling at $\geq 40\text{Hz}$.	Rule 3.2	The maze floor contains distinct colors. (black, blue, ...) Requires high speed bottom facing color sensors.
REQ-05 Navigation	Multi-floor mapping with autonomous home return and smart routing	Rule 5.7	To maximize exploration points, the robot must map unknown layouts, handle multi-floor ramps, and reliably navigate back to the starting tile, while determining the best route for the highest success rate.

Project Schedule and Review Gates

The project followed a strict sequential-parallel pipeline. Hardware iterations decoupled software testing through early simulation. Tasks were managed via Microsoft Teams, reviewed in weekly mentor meetings.

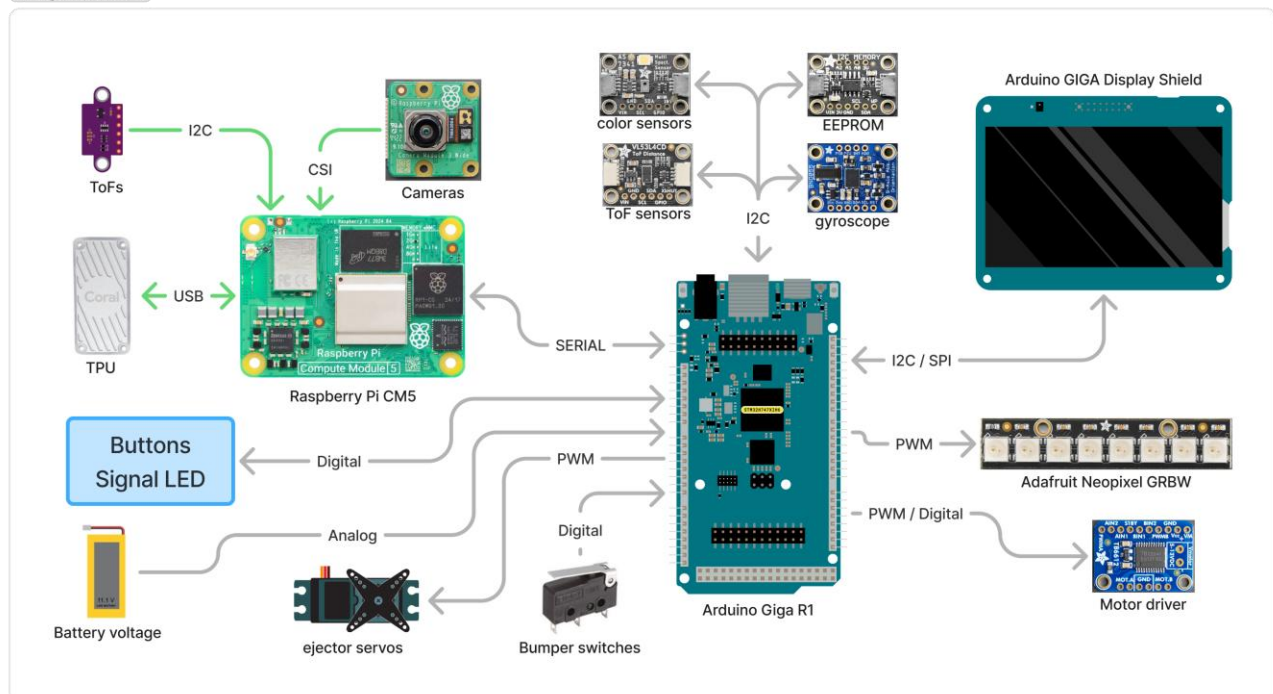


Gate	Focus	Criteria
Gate 1	Design and Algorithm validation	3D CAD models evaluated in Autodesk Fusion for precise internal component clearance - Mapping clears 20 randomized maze configurations without failure
Gate 2	Electronics and component lock	- Schematic and PCB layout approved for manufacturing via DRC check rules and manufacturer - Camera communication with Arduino Giga R1 validated - Non-Blocking I2C sensor readout validated via time loop analysis
Gate 3	Full system integration	- Fully assembled robot operates autonomously for an 8-minute scoring run - Chassis successfully clears standard 2cm speed bumps - Vision System triggers correct victim marking routines via UART within $\leq 1s$ of detection

b) Integration Plan

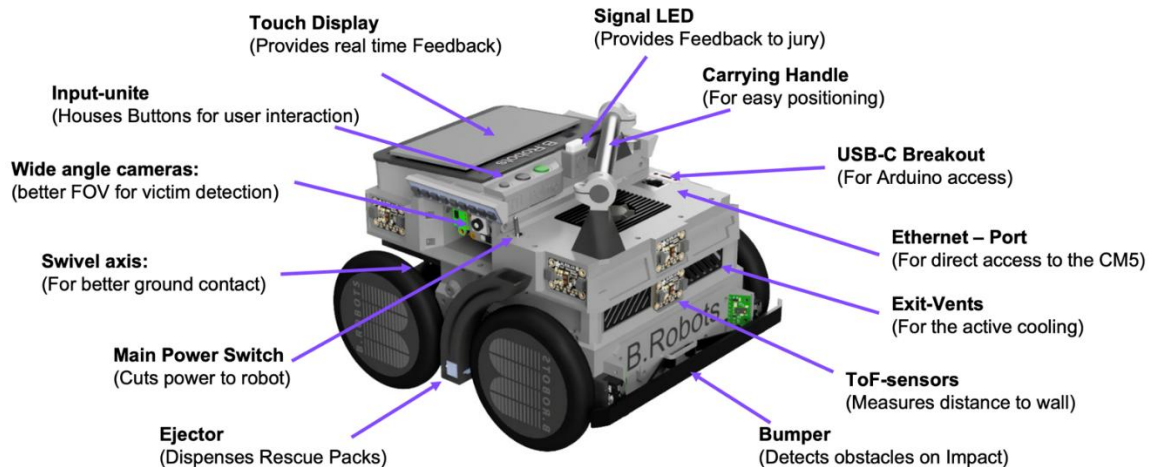
- How we integrated our solution:
 - > VL53L4CD ToF sensors distance measurement and wall detection
 - > BNO055 9DoF IMU measures current angle of the robot
 - > AS7431 spectrum sensor detect floor color
 - > Motor encoder get driven distance
 - > Bumper (micro-switches) Collision detection
 - > RasPi cameras Victim detection
- We must analyse data from various sensors like ToF sensors to let the robot drive in a straight line or to turn we use a gyro sensor, and the color sensors detect the tile color.
- The camera also needs to be positioned in a specific angle to cover as much area as possible.
- Communication between components:

Integration Plan



3) Hardware

Our robot features a modular hardware design that combines a robust mechanical structure with powerful electronics. A custom 3D-printed chassis, modular subsystems, and the Inter-Layer Connector system allow for fast maintenance and easy upgrades. Navigation is handled by an Arduino GIGA R1, while a Raspberry Pi CM5 performs real-time victim detection. Together with multiple sensors, custom-designed PCBs, and active cooling, the hardware provides a reliable and high-performance platform for autonomous rescue operations.



a) Mechanical Design and Manufacturing

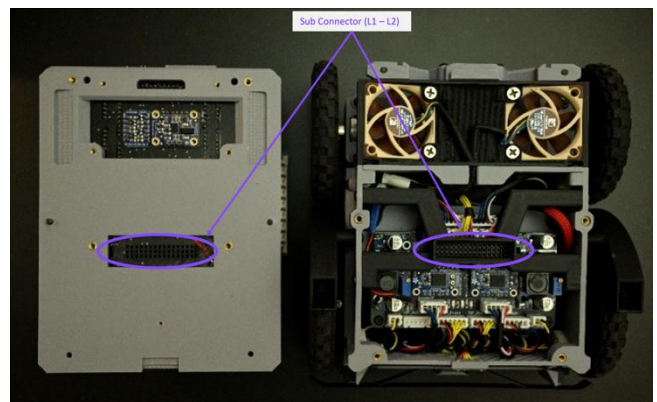
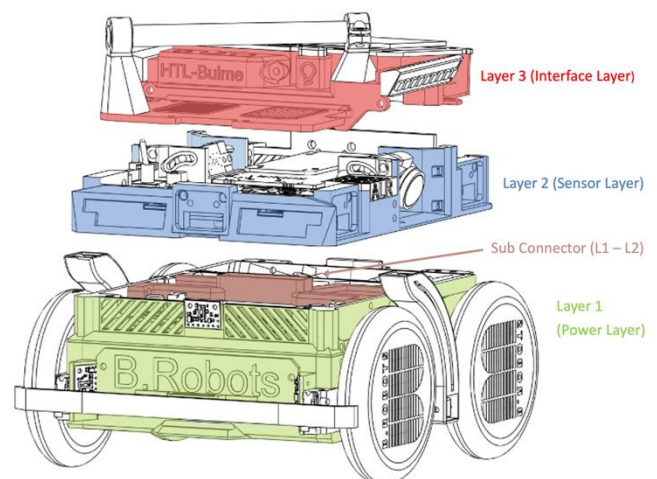
Inter-Layer-Connector

One of the most innovative and practical upgrades we implemented in this year's robot design is what we call the **"Inter-Layer Connector."** Traditionally, wiring between stacked layers of a robot such as sensor boards, control units, and power distribution boards can become messy, difficult to manage, and prone to disconnecting or wear during maintenance. To solve this, we developed a system where all electrical connections between the layers are routed through **permanently mounted pin headers**.

These connectors serve as **rigid, standardized interfaces** between each modular layer of the robot. This means that layers **can be easily detached** from one another without the need to unplug individual wires or trace connections manually. Whether we need to **swap out a board**, troubleshoot a malfunctioning sensor, or perform routine maintenance, the process is significantly faster and **more reliable**.

Another major advantage of this system is that it eliminates the need for loose jumper wires or ribbon cables running between levels. This not only improves the mechanical integrity of the robot but also results in **much cleaner and more professional cable management**.

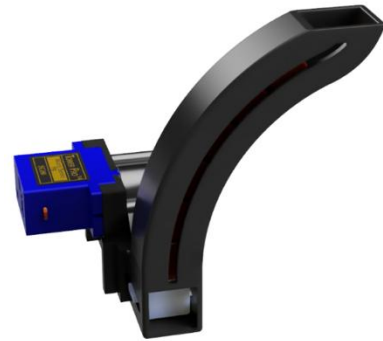
Ultimately, the Inter-Layer Connector enables us to disassemble the robot down to nearly any individual component without unplugging a single wire. It streamlines both the development and maintenance processes, **saves time during critical repairs**.



Modular Rescue-Pack-Ejector

Modularity is a key design principle for our team, which is why we developed the **Res-Pack-Ejector module** to be fully modular and easily replaceable. This allows for quick maintenance and minimal downtime in case of mechanical issues—individual ejectors can be **swapped in under 10 minutes**.

Each ejector uses a 9g servo motor to drive a gear connected to the piston. The **hopper is curved** to follow the outer radius of the front wheels, optimizing the camera's field of view. Its width ensures that rescue packs drop into place smoothly **by gravity**, without getting stuck.



Swivel axis

After seeing the problems other teams had in past competitions, we noticed that many struggled with obstacles on the ground. Their robots didn't always keep full **contact with the floor**, which caused issues with sensor readings.

To solve this, we built a swivel axis that can **rotate freely** using two ball bearings. It adjusts **automatically to the ground**, helping our robot stay stable.

Thanks to this design, all four wheels stay in **contact with the ground at all times**, making movement smoother and sensor data more reliable. The components for this mechanism we custom machined on a lathe from 15mm stainless steel rods.



Active cooling

After being faced with overheating issues we decided to **implement active cooling** into our robot.

We arranged two Noctua fans to blow air via **air channels** through our robot constantly. This not only prevents heat caused bottlenecks but also **adds to the design** of the robot.



Manufacturing

The **chassis** has been **custom-designed** in **Fusion 360** and **3D-printed** in various materials (PLA, PLA-CF and ABS), depending on their function and application.

Testing

Critical **subsystems** such as the **power system**, **rescue pack ejector** and **motors** have undergone rigorous testing to prevent any failures during the competition.

Rescue pack ejector: Has been tested with hundreds of packages using an extended hopper.

Motors: The motors have been run at various speeds inside and outside the robot to ensure the temperature does not exceed the softening point of the 3D-printed casing.

b) Electronic Design and Manufacturing

ToF (Time of Flight)

The Adafruit **Time of Flight sensors** manage multiple tasks such as **determining the distance** to the walls, checking for surrounding walls or **detecting obstacles** like "stairs" or bridges. We use a **total of 10 Time of flight sensors** arranged around our robot.



Gyro

We use the Adafruit BNO055 Gyro breakout board for determining the **absolute rotation** and orientation (North, South ...) of our robot. It manages tasks like turning the robot and can detect obstacles like **ramps, stairs and Bumpers**. When driving up a ramp it is also responsible for determining the levels the Robot is on.

Color-Sensor

For detecting black, blue and silver tiles we use two **Adafruit AS7341 10-Channel Light / Color Sensor**. It is located on the **bottom front and the middle** of the lowest layer. The calibration process is done via the display. The Adafruit AS7341 10-Channel Light / Color Sensor doesn't detect RGB values **but 10 different wavelengths of reflected light**.

Main-Navigation-Controller:

We chose the **Arduino Giga R1** as our robot's main processor because it retains the familiar **Arduino infrastructure** while offering more **Digital I/O** pins than the Arduino MEGA. Its **dual-core STM32 architecture (480 MHz main / 240 MHz secondary)** significantly outperforms the MEGA's 16 MHz, enabling faster pathfinding and a responsive touch display. Additionally, the built-in **JTAG port** provides seamless debugging in Arduino IDE 2. We also plan to leverage the onboard Wi-Fi and Bluetooth for remote **serial monitoring and** over-the-air code uploads. It handles all driving related tasks including all sensor readings and the mapping.

Vision Controller:

For the camera detection we use a **Raspberry CM5** compute module, which runs **Raspberry OS**. This single board computer handles the **Victim detection** via the side-cameras. Its 2.4 GHz quad-core architecture vastly outperforms traditional microcontrollers, easily handling real-time computer vision. The exposed **PCIe and MIPI interfaces** allow seamless integration of high-speed storage and cameras. We will use the **onboard Wi-Fi and Bluetooth for remote desktop**.

PCBs:

All **PCBs**, except for various breakout boards, have been custom designed in **EasyEDA-Pro** a software to design custom PCB schematics and layouts which are **manufactured by JLCPCB**. The final assembly, including component soldering and mechanical construction, was performed by hand.

Bottom PCB (power layer):

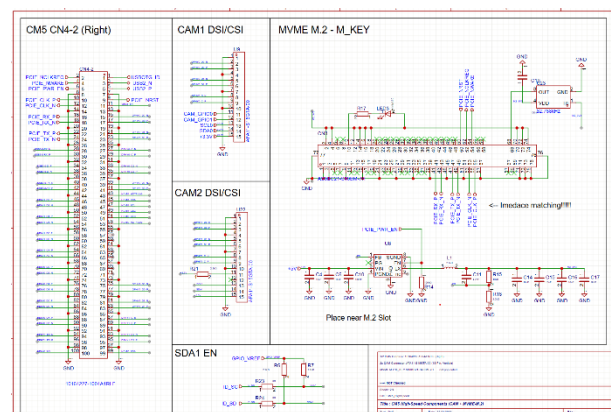
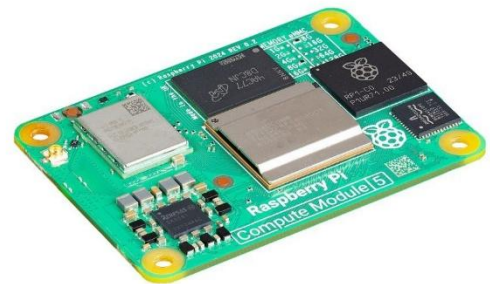
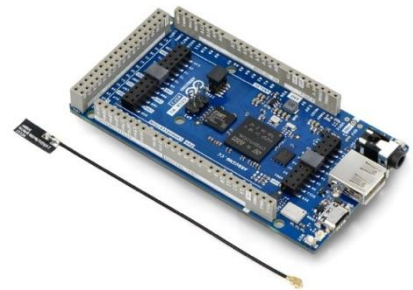
Connects all high-current applications, such as motor drivers, servos and step-down converters.

The middle PCB (signal layer):

Houses the CM5, Arduino and almost all sensors. Our custom PCB also features an Ethernet port for direct access to the CM5, as well as an M.2 SSD slot for additional storage or future expansion.

Top PCB (Interface Layer):

Connects the display to the Arduino and handles the input and output of the signal unit.



4) Software

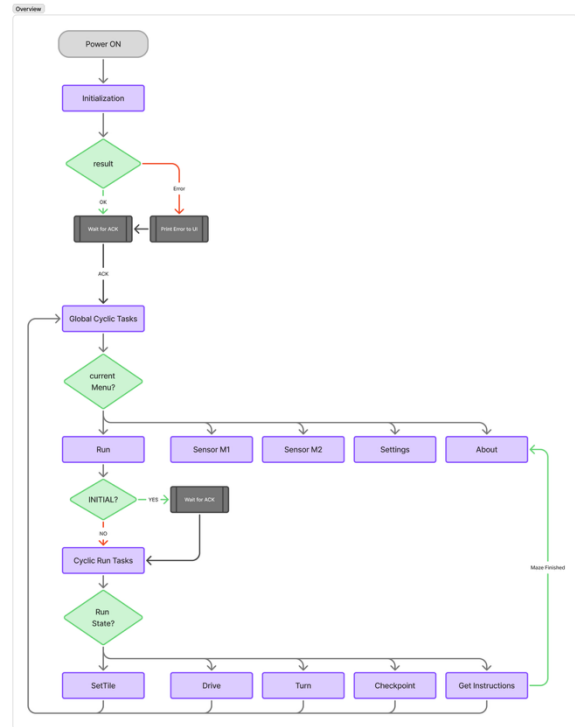
The robot firmware is written in C++ and built with PlatformIO using Visual Studio Code. The repository is hosted on GitHub, where a continuous-integration pipeline (GitHub Actions) compiles every commit, guaranteeing that the main branch always builds. Three design principles run through the entire codebase:

- **strict modularity:** every subsystem is an independent library with a small public interface
- **a uniform API:** all modules communicate through shared data types and a common `ErrorCodes` return type, making interfaces predictable and errors raceable
- **non-blocking execution:** no module is allowed to stall the control loop

a) General software architecture

The architecture is built on two nested state machines. The outer `RobotState` machine handles the operating mode (boot, settings, sensor info, calibration, run), while the inner `RunState` machine executes the competition run as a **deterministic cycle**: `SETTLE` → `GET_INSTRUCTIONS` → `TURN` / `DRIVE` / `RAMP` and back. Every loop iteration first executes the global cyclic tasks (sensor updates, UI); during a run, additional cyclic run tasks (camera polling, floor-color and bumper handling) are layered on top. This separation guarantees that sensor data is always fresh, independent of the current run phase.

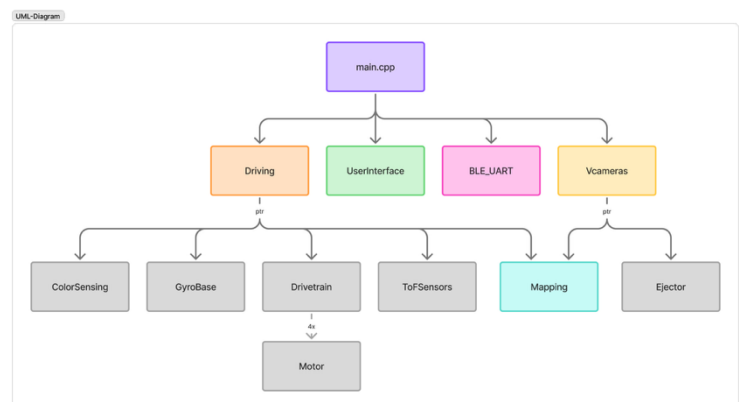
Each subsystem is realized as its own C++ class inside a dedicated library. Dependencies are wired explicitly by pointer injection during initialization. This removes hidden global singletons, so every interaction path is visible in the UML diagram (see UML diagram below) and each module can be tested in isolation.



The data flow between the modules forms a clear pipeline: `TofSensors` condenses four time-of-flight readings into a wall bitmask; `Mapping` merges it with the floor type from `ColorSensing` into its tile graph and emits a high-level Instruction set (turn / drive / ramp); `Driving` translates each instruction into closed-loop motion using the IMU and the ToF sensors, and commands the `Drivetrain`. Victim handling runs in parallel: `Vcameras` receives classifications over UART and directly triggers the `Ejector`.

Integration problems solved in the software:

all VL53L4CD sensors share one I²C address at power-up. The firmware sequentially enables them via their XSHUT pins and reassigns unique addresses. The camera link uses a compact UART protocol — the microcontroller receives only the final classification (side, victim type), keeping the interface trivial to test.



UML class diagram – module structure and interaction paths
(ptr = pointer injection)

b) Innovative solutions

A* pathfinding with a weighted cost model and adaptive re-routing

Navigation is based on an A* search over a 3D tile graph (x, y, z for multiple maze levels). The map is statically allocated for up to 256 tiles which leads to no heap usage, therefore fully deterministic memory behavior on the microcontroller. Every tile carries a traversal cost: regular 1, ramp 5, blue 7, danger zone 8. The planner therefore avoids expensive tiles automatically whenever a cheaper route exists, instead of relying on hard-coded rules. Exploration targets are selected by the same search, so exploring and returning home use one consistent mechanism. The cost model also drives obstacle handling: when a bumper is triggered, the current tile's cost is increased (or the route is locked entirely) and the active path is invalidated. The A* immediately plans around the obstruction on the next cycle. A blocked corridor therefore degrades gracefully into a detour instead of a stuck robot.

```
Instructions Mapping::GetInstruction(){
    if path is finished OR bumper was triggered:
        //Finish run
        if return_home is active AND currentPosition == start:
            return MazeFinished

        // reset Bumper Flag
        bumper was triggered = false

        // find next target position with BFS
        target = findNextTarget()

        // calculate cheapest path to target with A*
        path = AStar(currentPosition, target)

        if target is unreachable:
            return unreachable

        // convert path to instructions
        convert path to instructions and save in path
        store instructions in path
        set pathIndex = 1
        return first instruction from path
    else:
        return next instruction from path
}
```

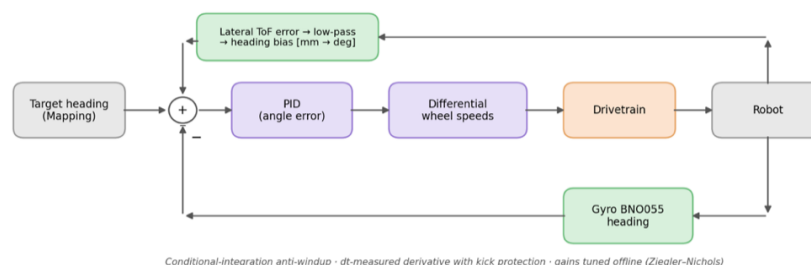
Checkpoint snapshot system

On every visited checkpoint the complete map state is snapshotted into a backup buffer. After a lack of progress, a single button press restores the map and position from the snapshot, overrides the orientation (the captain replaces the robot facing a known direction) and clears the active path so A* replans from clean data. A reset counter adds strategy: after a configurable number of resets the planner is forced into return-home mode to secure the exit bonus rather than risking further failed exploration.

Advanced PID drive and turn control

Straight driving is controlled by a PID controller whose single feedback signal is the gyro heading. The lateral wall offset measured by the side ToF sensors is low-pass filtered and converted into a small *bias on the target heading* (mm → degree). The controller therefore always operates on one physical quantity; the integral term stays meaningful, unlike other implementations that mix millimeters and degrees in one error signal. The controller runs with a measured time step dt (clamped to suppress derivative kick after long gaps) and conditional-integration anti-windup: the integral only accumulates while the output is unsaturated. P/I/D constants were tuned offline using the Ziegler–Nichols method. At ToF-unreliable positions (intersections, openings) the lateral correction is suppressed, and the gyro alone holds the heading.

Turning uses a separate PID instance on the heading error with case-specific speed limits (reduced for 180° turns and while a camera alert is active) and a timeout as a safety net. After each turn the robot re-aligns to the nearest wall and snaps its heading to the cardinal orientation, preventing gyro drift from accumulating over a run.



Victim detection system on the Raspberry Pi 5

A Raspberry Pi 5 Compute Module processes dual wide-angle camera streams in real-time via isolated threads. For the purpose of character recognition (Φ , Ψ , Ω), grayscale frames are scaled and evaluated by TensorFlow Lite using a grid pattern. In the absence of a detected letter, an HSV (Hue, Saturation, Value) color scan is employed to analyze perspective-warped ring targets, thereby determining the health status of the victim. The

validation of a statistical filter is contingent upon five consecutive identical detections, which are supported by a 3-second watchdog. The results that have been validated are transmitted via UART to an Arduino master that controls the camera states.

Further innovations:

- **Non-blocking sensor framework:** Modified ToF library with continuous ranging and data-ready polling instead of blocking reads. This is the key enabler of the 23 ms loop time evaluated in C below.
- **Continuous Traversal:** The robot does not stop on each tile when driving straight. In the middle of each tile, information about walls, the floor, and ramps is saved and sent to the mapping system. The robot only stops if a wall is in front of it or if the mapping commands a turn.
- **Live map on the touchscreen:** The internal map is rendered robot-centered and rotates in real time during the run. This feature dramatically speeds up debugging of mapping errors.
- **In-place spectral calibration:** The 10-channel AS7341 floor sensor is calibrated directly at the venue via the touchscreen. Calibration data is stored in EEPROM, which makes floor classification robust against venue lighting.
- **Offloaded vision:** Neural network inference runs entirely on the camera side. The microcontroller only receives the final classification via UART. The mapping module suppresses duplicate detections on the same tile.

c) Reliability, testing and quality assurance

Rather than relying solely on post-assembly benchmarks (see Section 5), our quality assurance pipeline focuses on structural and architectural validation throughout the entire development process.

- **Continuous Integration & Debug Flags**

The GitHub CI pipeline enforces automated compilation and static code analysis with every remote commit to prevent syntax and architectural regressions. For modular hardware testing, custom compile-time flags (`PID_TUNE_MODE`, `TURN_TUNE_MODE`, and `RAMP_TEST_MODE`) isolate specific subsystems from production code paths. This ensures the calibration of individual control loops is isolated.

- **Deterministic execution & Implemented Diagnostics**

To ensure the critical loop runtime requirement (REQ-03, ≤ 25 ms) is met, all software components use non-blocking, flag-based sensor polling instead of standard delays. For in-depth runtime analysis, critical classes have an optional diagnostic port interface. A pointer to a *Stream* object (e.g., `&Serial` or `&BLE`) can be passed to the constructor. If this is omitted (`nullptr`), all internal logging structures are bypassed, which eliminates runtime and memory overhead during official competition runs.

- **Component Matching & Drift Mitigation**

Software reliability depends heavily on uniform hardware behavior. To prevent systematic odometry drift within the dual-drive PID loop, we individually characterized all Time-of-Flight (ToF) sensors on a dedicated calibration rig. The firmware compensates for minor manufacturing tolerances by mapping and matching identical distance response curves before the robot ever enters a physical environment.

5) Performance evaluation

a) Reliability Testing and Quality Assurance

To verify operational robustness under competition constraints, our team implemented a rigorous testing pipeline:

- **Software Simulation:** The rewritten mapping system was stress-tested across 20 randomized maze configurations within a custom-developed simulation environment prior to physical roll-out, achieving a 100% exploration success rate. (perfect inputs)
- **Physical & Telemetry Testing:** We built a custom Test Maze, utilizing real-time diagnostics from the Serial Bus and synchronized video analysis to systematically identify and minimize performance errors.
- **Stress & Endurance Runs:** In accordance with our review gates, the full system was validated through continuous 8-minute autonomous scoring runs and mechanical clearance tests over standard 2cm speed bumps and particulate surface hazards (e.g., scattered corn kernels and gravel), ensuring uninterrupted traction and chassis stability.

b) Module Difficulties and Engineering Fixes

During system integration and testing, two specific modules caused significant technical challenges. They were systematically analyzed and solved:

Main Loop Latency:

- > Problem:
During autonomous runs, the main control loop suffered periodic latency spikes exceeding 200ms (up from a 30ms baseline). This was primarily caused by updating the live grid map on the touch display every 50ms. Transferring the map's graphical data was heavily bottlenecked by the transmission speed (SPI/I2C bus bandwidth) to the Arduino GIGA Display Shield, which blocked the control loop during data transfer. Consequently, sensor polling was delayed, causing the robot to miss black floor tiles (violating REQ-04) and inducing severe PID driving instabilities (violating REQ-03).
- > Solution:
We refactored the UI logic so that the map is redrawn only within the SetTile routine when new mapping information is registered. By the time this routine is executed, the robot is already safely positioned in the center of a tile, rendering the bus transmission delay harmless to active driving functions. Additionally, we implemented non-blocking, asynchronous polling for all time-of-flight (ToF) and color sensors, permanently stabilizing the baseline loop time at 23ms.

Driving

- > Problem:
When traversing multi-floor ramps or stairs, wheel slip prevented the accurate calculation of travel distance using standard motor encoder data alone. Additionally, the robot's kinematic behavior and slip characteristics varied significantly depending on the obstacle profile (e.g., smooth ramps versus stepped stairs) and direction of travel (e.g., uphill versus downhill). This resulted in severe localization errors in the global mapping system.
- > Solution:
We developed a dynamic calibration algorithm that logs real-time pitch data from the BNO055 IMU and raw distance steps from the motor encoders throughout the incline. After completing the traversal, the system calculates the average slope inclination and verifies the plausibility of the path using the total elapsed execution time. To reconstruct the precise tile length, the algorithm applies a correction coefficient empirically determined through rigorous testing for each obstacle type and driving direction permutation. This ensures accurate multi-floor mapping integrity.

6) Conclusion

In conclusion, B.Robots Seniors built a smart, reliable, and easy-to-maintain robot that performs very well in the Rescue Maze competition.

7) References

a) Software

[Visual Studio Code](#)

[PlatformIO](#)

[GitHub](#)

[Arduino](#)

b) Hardware

[Autodesk Fusion](#)

[EasyEDA](#)

c) Manufacturing

[JLCPCB](#)

[Bambu Lab](#)

d) Planning

[Microsoft Teams](#)

e) Diagrams

[Figma](#)